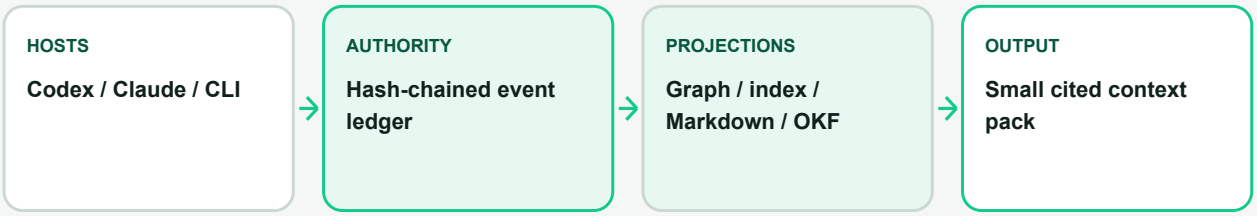
TECHNICAL WHITE PAPER

Qarinah: Less Context. More Proof.

An evidence-linked project-memory compiler for coding agents



Ajnas NB

Paper version 1.0 - July 2026
Qarinah 0.1.0-alpha.3 technical preview
Apache License 2.0

Status: Implementation-backed technical white paper. Qarinah is currently in alpha, and the paper has not undergone independent peer review.

Contents

This paper describes the implemented architecture, evidence model, security boundary, evaluation, and release status of Qarinah.

Abstract

Long-running software work creates a memory problem for coding agents. Source files show what the project contains now, but they often do not explain why a design was selected, which alternative was rejected, who approved a consequential action, what a tool returned, which source supported a claim, or whether a later decision superseded an earlier one. Replaying every prior interaction preserves more history, but context volume grows with project age and introduces increasing amounts of irrelevant material. Replacing that history with a single generated summary is smaller, but it weakens provenance and makes omissions, conflicts, and obsolete decisions difficult to inspect.

Qarinah treats project memory as a compilation problem rather than a transcript-storage problem. Its authoritative layer is an append-only, hash-chained JSONL ledger containing only explicitly permitted lifecycle events and committed decisions. From that ledger it deterministically derives a typed graph, a lexical and typo-tolerant index, human-readable Markdown, a bounded project-structure projection, and portable Google Open Knowledge Format (OKF) Markdown. At task time, a hybrid retriever compiles a budgeted context pack whose items cite the exact event identities and hashes that support them.

The implementation is local-first, model-agnostic, and explicit about capture. A repository configuration does not grant consent by itself. Capture also requires machine-local trust for the repository's real path, and metadata-only capture is the default. Content capture is a separate reviewed choice. Derived files are disposable and can be rebuilt from the verified event chain.

The committed software-task evaluation compares full-history replay with Qarinah packs while keeping the same current-task source material on both sides. Across 240 retained records and six software scenarios, the measured context falls from 442,113 to 5,682 estimated input-context tokens, a 98.71% reduction and a 77.81:1 compression ratio. Every required target ranks in the top five with direct evidence coverage, and no model-written summaries are used. A separate fixed-budget long-document evaluation preserves all 16 supported answers at rank one, rejects all four unsupported controls, and achieves at least 98.4% estimated context reduction. These are reproducible context-volume and retrieval results, not universal claims about provider billing, task quality, latency, or coding speed.

This paper describes the problem, system model, architecture, trust boundary, retrieval method, integrations, evaluation, limitations, and release criteria of the current implementation.

Keywords: coding agents, project memory, context engineering, provenance, knowledge graph, retrieval, audit log, MCP, Codex, Claude Code, governance

1. Executive summary

Qarinah is designed around one observation: useful agent memory needs both compression and proof.

A coding agent should not have to reread months of unrelated project history to recover one release decision. It should also not be asked to trust an uncited paragraph that may have omitted a constraint or preserved an obsolete choice. Qarinah keeps the durable evidence record separate from the compact context sent to a model:

1. **Capture is explicit.** A workspace is initialized, reviewed, and trusted on the current machine before Qarinah records anything.
2. **The ledger is authoritative.** Events are canonical, versioned, append-only, hash-chained, bounded, and linked to provenance.
3. **Every other view is derived.** The graph, index, Markdown record, project map, OKF bundle, and context pack can be deleted and rebuilt.

4. **Retrieval is evidence-aware.** Lexical relevance, typo tolerance, graph evidence, conflicts, supersession, time, retention, authority, and diversity influence selection.
5. **Output is budgeted as a complete artifact.** Qarinah measures the full serialized pack, including citations and coverage metadata.
6. **Selected memory remains inspectable.** Every returned item identifies the event and hash that support it.
7. **Governance remains composable.** Maqam can govern registered context reads and approved writes; Cockroach Crawler can provide public source evidence; ProductLoop can provide workflow provenance.

The project does not claim to read hidden reasoning, monitor every host action, prove the truth of a stored claim, or turn a user-space package into an operating-system security boundary. It provides a narrower and more useful primitive: a verifiable local record and a deterministic way to compile the smallest evidence-backed context appropriate for a task.

2. The project-memory problem

2.1 Source code is necessary but incomplete

A repository contains the executable state of a software project, but engineering work also depends on information that may not live in the current code:

1. the reason a migration was split into phases;
1. an approval that authorized one release artifact and not another;
1. a production failure and the tool output that identified its cause;
1. a source used to justify an implementation choice;
1. a rejected approach that must not be reintroduced;
1. a security boundary agreed during review;
1. a previous decision that a newer decision superseded; and
1. the relationship between a workflow run, its artifacts, and its release receipt.

When this context is absent, a new agent can read the code correctly and still repeat an old mistake.

2.2 Full-history replay scales poorly

The direct solution is to resend all retained history. That has three weaknesses.

First, context volume grows with the lifetime of the project, not with the needs of the current task. Second, relevant evidence competes with unrelated history for model attention. Third, sending more history increases the disclosure surface: material unrelated to the task is exposed simply because it exists.

Full replay is valuable as an audit baseline, but it is a poor default context strategy.

2.3 A single summary is not an authority

A generated summary can be compact, readable, and useful. It is still a lossy interpretation. If it becomes the only retained artifact, a later reader may be unable to answer:

1. Which source supported this sentence?

1. Was the statement extracted, inferred, claimed, or verified?
1. Was a conflicting decision removed or merely omitted?
1. Is this decision still current?
1. Did the summary cross a capture or retention boundary?
1. Can the result be reproduced without calling the same model?

Qarinah may retain summaries as events, but it never makes a summary authoritative merely because it is concise.

2.4 Similarity is not evidence coverage

Semantic or lexical similarity can identify related material without showing that the requested evidence is present. A result about "database rollout" may be close to a query about "orders idempotency migration" while still omitting the exact decision the task requires.

For this reason, Qarinah reports coverage separately from ranking. A highly ranked result can still fail a caller's `minimumCoverage` requirement.

3. Goals, non-goals, and contribution

3.1 Goals

The current system is built to:

1. preserve permitted project events in a durable local record;
1. make alteration, deletion, truncation, duplication, and rollback detectable relative to a machine-local checkpoint;
1. keep capture consent separate from portable repository configuration;
1. distinguish metadata capture from reviewed content capture;
1. reconstruct all search and presentation views deterministically;
1. preserve typed relations, conflicts, supersession, provenance, confidence, time, authority, and retention;
1. compile small context packs under explicit character, item, and token budgets;
1. return citations to the exact retained evidence selected;
1. integrate with Codex, Claude Code, local CLIs, MCP diagnostics, Maqam, Cockroach Crawler, ProductLoop, and OKF without silently merging their security boundaries; and
1. remain useful without a hosted Qarinah account, Qarinah API key, remote database, or background service.

3.2 Non-goals

Qarinah does not:

1. capture private model reasoning or chain of thought;
1. scrape hidden transcript files, browser cookies, or authentication state;

1. guarantee observation of every action available in every AI host;
1. execute retrieved text as instructions;
1. prove that a retained claim is true because its bytes are hash-chained;
1. promise secret-free content capture for arbitrary tool output;
1. provide universal semantic memory or automatic answer correctness;
1. replace a model provider, agent orchestrator, database, source-control system, or privileged sandbox; or
1. claim operating-system control over unregistered processes, direct side effects, devices, identities, or network traffic.

3.3 Technical contribution

The implementation combines several properties that are often separated:

1. a machine-consent boundary tied to the repository's real local path;
2. a versioned, canonical, append-only project event model;
3. a hash chain plus a machine-local rollback checkpoint;
4. disposable graph, index, Markdown, project-structure, and OKF projections;
5. deterministic hybrid retrieval with visible conflict and supersession behavior;
6. complete-output context budgeting and explicit evidence coverage;
7. thin host adapters that do not claim more observability than their hosts expose; and
8. optional governance and source-ingestion adapters that keep ownership and authority explicit.

The important distinction is not that Qarinah contains a graph or a search index. It is that those views remain subordinate to a verifiable event record and are compiled into bounded, cited task context.

4. System model

4.1 Actors

Qarinah events distinguish five actor classes:

1. **human** for an explicit user decision, approval, or statement;
1. **agent** for an exposed model or subagent lifecycle event;
1. **tool** for a tool request, result, artifact, or failure;
1. **system** for an adapter, runtime, or deterministic system transition; and
1. **source** for externally acquired evidence.

An actor class describes provenance. It does not automatically grant authority.

4.2 Event classes

The current contract represents prompts, tool requests, tool completions, approvals, artifacts, sources, claims, decisions, summaries, compactions, subagents, completed turns, and failed turns when a supported adapter supplies them. Unknown lifecycle events are not guessed into a familiar shape.

Each retained event contains:

1. a schema version;
1. a workspace identity;
1. an event identity;
1. optional session and turn references;
1. an actor;
1. a canonical UTC timestamp;
1. an event kind;
1. bounded content or metadata;
1. provenance;
1. a confidence class;
1. typed relations;
1. retention metadata;
1. a previous-record hash;
1. a content hash; and
1. a canonical record hash.

4.3 Confidence, authority, and truth

Qarinah separates four confidence classes:

1. `extracted`: copied or mechanically derived from a named source;
1. `inferred`: produced by a reasoned transformation;
1. `claimed`: asserted but not independently verified; and
1. `verified`: checked against an explicit verification process.

These labels communicate how a record entered the system. They do not turn Qarinah into a fact-checker. A verified hash proves consistency with retained bytes; it does not prove the world described by those bytes.

Scoped authority is also independent. A human approval can be authoritative for one run, tool, input, or release artifact without becoming a universal permission.

4.4 Relations

Typed relations connect events to sessions, turns, tools, approvals, sources, evidence, artifacts, conflicts, and superseded records. They allow retrieval to distinguish "this record supports that decision" from "this record conflicts with that decision."

This matters because project history is not merely a bag of text. Its topology carries meaning.

5. Trust and capture

5.1 Portable configuration is not consent

A file committed to a repository can travel to another machine. It must not silently authorize capture there. Qarinah therefore requires two independent elements:

1. portable workspace configuration in the repository; and
2. a machine-local permit bound to the canonical real path and the reviewed policy digest.

The permit records the workspace identity, enabled state, capture mode, event, log and context ceilings, retention class, verified head, and disposable event-ID projection digest. If the portable policy changes, capture fails closed until the new digest is explicitly reviewed and trusted.

5.2 Metadata-first retention

Metadata mode is the default. The central append boundary replaces unreviewed bodies and data with deterministic digest and size information. Built-in adapters may retain only code-reviewed coarse metadata fields.

Content mode can retain bounded prompt, tool, completion, source, and decision content after best-effort redaction. It is intentionally a separate choice because content useful to future agents can also be sensitive.

5.3 What Qarinah excludes

The product boundary excludes:

1. credentials and environment values;
1. browser cookies and private session state;
1. hidden reasoning;
1. ignored source-file contents;
1. unrequested transcript scraping; and
1. unrelated files outside the opted-in repository root.

Redaction handles secret-like keys and common token patterns, but no pattern-based filter can guarantee that arbitrary content is free of secrets. Metadata mode is the safer default for unclassified work.

5.4 Revocation

A machine-local revocation tombstone takes precedence over portable configuration and trust-record recreation. Re-enablement requires an explicit trust operation. Disabling, untrusting, re-trusting, appending, and updating checkpoints serialize through the workspace write lock.

6. Authoritative storage

6.1 Canonical JSONL

The durable source of truth is:

```
.qarinah/events/events.jsonl
```

Each line is a canonical event envelope. Canonical serialization ensures that logically identical JSON values have one byte representation before hashing.

6.2 Hash chaining

For event \(E_i\), Qarinah binds the canonical event content and the previous event hash:

```
recordHash(E_i) = SHA-256(canonical(E_i without recordHash) + previousHash)
```

The exact implementation uses the package's versioned canonicalization and event contract rather than accepting arbitrary JSON. The chain makes edits, deletions, truncation, duplicate identities, non-canonical bytes, and broken continuity detectable during full verification.

6.3 The rollback checkpoint

A hash chain alone cannot identify the correct head if an attacker replaces the complete log with an older valid prefix. Qarinah therefore stores the last trusted head in the machine-local permit. Verification compares the retained chain with that checkpoint.

This is tamper-evident rather than tamper-proof. An adversary able to replace both repository state and machine trust state remains outside the current foundation. Signed and independently anchored checkpoints are roadmap work.

6.4 Concurrent append behavior

Appends use a renewable owner-token lease. Under the lock, Qarinah:

1. reloads machine trust;
2. validates the canonical head;
3. validates the checkpoint-authenticated event-ID projection;
4. extends and flushes the event log;
5. updates the disposable idempotency projection;
6. checkpoints both identities; and
7. releases the lock only if ownership remains unchanged.

The initial coordination design is for one host and one workspace. Network filesystems require a different consensus and locking model.

6.5 Idempotency and collisions

Stable event identities let exact replays return the retained record rather than duplicate it. If the same event identity is reused for different canonical content, the append fails with a conflict instead of silently forking history.

This behavior is especially important for workflow ingestion and source acquisitions, where retries are normal.

7. Deterministic derived views

The event chain is authoritative; the following paths are rebuildable:

Path	Purpose	Authority
`.qarinah/graph/graph.json`	Typed event graph and current project-structure projection	Derived
`.qarinah/index/index.json`	Lexical postings, trigram terms, and graph adjacency	Derived
`.qarinah/records/CONTEXT.md`	Human-readable project record	Derived
`.qarinah/records/okf/`	Portable OKF Markdown bundle	Derived
`.qarinah/index/event-ids/`	Checkpoint-authenticated idempotency buckets	Disposable
`.qarinah/objects/`	Reserved content-addressed source snapshots	Reserved
`.qarinah/snapshots/`	Reserved signed context-pack manifests	Reserved

A build starts by verifying the complete event chain and checkpoint. Derived state is then recomputed from canonical inputs. Retrieval rejects stale persisted views rather than quietly using them.

This design offers two practical benefits. A corrupt search index does not become durable memory, and a human can inspect the Markdown or OKF representation without changing the evidence record from which it came.

8. Project-structure observation

Qarinah can scan a repository to create a bounded structural snapshot. The scanner records:

1. directories and files;
1. portable paths;
1. file sizes and content identities;
1. conservative JavaScript and TypeScript module references;
1. Markdown links;
1. exact observed source spans for extracted references; and
1. additions, changes, renames, and deletions relative to the prior snapshot.

The scanner follows the project's ignore policy, excludes generated Qarinah state, rejects linked paths, bounds file count, file size, total bytes, and traversal depth, and skips likely binary files.

The scanner does not claim to be a compiler, a language server, or a universal symbol graph. Its purpose is to answer structural questions such as:

1. Which files were affected by this decision?
1. Where is a referenced module located?
1. What changed between two recorded project snapshots?
1. Which artifact did a tool completion produce?

Deeper language-aware adapters can be added later without changing the authority of the event ledger.

9. Retrieval and context compilation

9.1 Candidate generation

Qarinah's local retriever combines:

1. BM25 lexical relevance;
1. character-trigram matching for bounded typo tolerance;
1. one-hop graph evidence;
1. reciprocal-rank fusion; and
1. deterministic diversity.

These methods do not require a model call or an embedding API. Optional future dense or model-assisted adapters must remain versioned, explicit, and subordinate to the authoritative record.

9.2 Evidence policy

Candidate relevance is only the first stage. Before selection, Qarinah applies:

1. `asOf` time filtering;
1. retention eligibility;
1. scoped authority;
1. conflict visibility;
1. supersession rules; and
1. diversity across the retained evidence.

An older decision can remain visible for audit while a superseding decision is ranked as current. A conflicting record is not silently flattened into the same conclusion.

9.3 Evidence coverage

Context-pack v2 reports:

1. `direct`: one retained record contains all normalized query terms;
1. `partial`: retained evidence overlaps the query but is incomplete; or

1. `none`: no matching retained evidence exists.

Callers can require `minimumCoverage: "direct"`. If the requirement is not met, compilation fails with `CONTEXT_COVERAGE_TOO_LOW`.

Coverage is deliberately conservative. It describes lexical evidence in the retained record, not semantic truth or the correctness of a model's eventual answer.

9.4 Complete-output budgeting

Qarinah budgets the entire serialized context pack, not only the selected excerpt. The limit includes:

1. item content;
1. event identities;
1. content and record hashes;
1. selection reasons;
1. evidence coverage;
1. manifest metadata; and
1. JSON or Markdown framing.

Callers can supply character and item ceilings or a deterministic token plan with maximum, reserve, citation, framing, and content budgets. Without a provider tokenizer, Qarinah uses the portable `ceil(characters / 4)` estimate and marks it as inexact.

9.5 Cited output

Each context item carries the retained identity required to trace it to the event log. A context pack is therefore a compilation artifact with a manifest, not an anonymous search response.

This is the central user-facing behavior: the next agent receives a small working set and can still inspect why every selected item was included.

10. Human-readable and portable records

10.1 Markdown and JSON

Qarinah materializes both machine-readable JSON and a human-readable `CONTEXT.md`. These views make the project record inspectable with ordinary tools and version-control workflows.

The generated Markdown is not intended to be pasted wholesale into every model task. It is an audit and navigation view. Task-time retrieval should compile a focused pack.

10.2 Google Open Knowledge Format

The explicit command:

```
qarinah export okf
```

creates a deterministic bundle compatible with the Google Open Knowledge Format 0.1 Draft. The bundle contains:

1. a root `index.md` declaring the OKF version;
1. a newest-first `log.md` grouped by event date;
1. one Markdown concept per event under `events/`;
1. bundle-relative typed relation links;
1. bounded citations and provenance; and
1. Qarinah extension fields for event identity, actor, confidence, authority, hashes, retention, and relations.

The output contains no export-time clock or destination-path field, so the same verified head produces the same bytes and bundle digest at every permitted destination.

Safe replacement requires a canonical ownership marker and expected manifest. Existing unmarked directories, unexpected files, path escapes, linked components, `.git`, and authoritative `.qarinah` locations are rejected.

OKF is a portable projection. Qarinah does not claim OKF ingestion or a lossless round trip, and OKF does not replace the ledger, graph, index, or query runtime.

11. Host integrations

11.1 One core, thin adapters

Qarinah keeps capture, storage, verification, derivation, and retrieval in one local core. Host adapters normalize only the fields their host explicitly supplies.

This avoids a dangerous compatibility pattern: inferring a universal private transcript format and pretending that every host exposes the same lifecycle.

11.2 Codex

The Codex plugin contains:

1. a standard `.codex-plugin/plugin.json` manifest;
1. ten supported local lifecycle hooks;
1. a Qarinah context skill;
1. a generated dependency-free runtime; and
1. a local read-only MCP diagnostics server.

Known event schemas reject missing or unrecognized fields. Unknown lifecycle event names are ignored rather than guessed. Local hooks do not claim coverage of hosted search or every specialized tool route.

Codex plugin installation is personal. The per-project Qarinah trust boundary controls which repositories may retain records.

11.3 Claude Code

The Claude Code plugin contains:

1. a native Claude plugin manifest;
1. hooks for sessions, prompts, tools, compaction, stop, subagent, and session-end events;
1. the same context skill workflow;
1. a generated dependency-free runtime; and
1. a local read-only MCP diagnostics server.

Qarinah does not parse Claude transcript files. Unknown hook fields are counted but their names and values are not retained in metadata mode.

The plugin requires the user to review the Node executable used by lifecycle hooks. Installed plugin caches are immutable snapshots; changes require a reviewed rebuild, reinstall, and reload.

11.4 CLI

The CLI supports explicit initialization, policy review, trust, append, scan, build, query, verification, diagnostics, and OKF export. Model-facing payloads can travel through bounded JSON on standard input so model-controlled text does not need to be interpolated into a shell command.

11.5 MCP

The bundled stdio MCP server intentionally exposes only:

1. `context_status`; and
1. `context_doctor`.

Both tools are read-only, closed-world diagnostics. They can select an exact opted-in workspace by absolute local path or `file:` URI. They never walk upward into a trusted parent, initialize a workspace, grant trust, mutate the ledger, repair a checkpoint, or disclose the absolute path in their response.

Context disclosure is not an ambient MCP side effect. A direct local query must be explicitly requested, or a separately governed Maqam capability can mediate it.

12. Composable ecosystem boundaries

12.1 Maqam

Maqam (documentation) is a TypeScript governance boundary for registered AI-agent operations. It evaluates policy before dispatch, can bind human approval to the exact run, tool, and input, consumes that approval once by default, and produces reviewable trace and evidence records. Qarinah remains a separate memory layer; Maqam can optionally govern exactly which Qarinah context reads and writes a host may perform.

Maqam can register two separate Qarinah tools:

Tool	Effect	Risk	Required authority
`context.query`	read	low	Active guarded dispatch and scoped evidence capability
`context.append`	write	high	Active guarded dispatch, scoped evidence, and consumed exact approval

The gateway verifier binds the active input and context objects, tool registration, run, canonical input hash, policy decision, and consumed approvals. A retained handler or fabricated plain context cannot replay that capability.

This governs registered calls. It does not mediate unregistered code, raw drivers retained by a host, or direct operating-system effects.

12.2 Cockroach Crawler

Cockroach Crawler (documentation) is a Node.js and TypeScript web-acquisition toolkit for AI agents. It crawls static and rendered pages, extracts normalized records, and routes supported public sources while keeping origins, credentials, browser capabilities, and resource ceilings under host control. Qarinah does not embed the crawler; it accepts reviewed `SourceRecord` values as source evidence through a strict interoperability boundary.

Qarinah validates a strict structural boundary around Cockroach Crawler `SourceRecord` values. Ingestion separates:

1. a stable content revision; and
1. a distinct acquisition containing retrieval and descriptive provenance.

An unchanged body fetched later can reuse its revision and append a new acquisition. Exact replay is idempotent. Metadata mode does not retain raw URL, title, text, author, warnings, or provider metadata. Content mode retains bounded reviewed values.

Crawler content remains untrusted evidence. Qarinah does not execute it and does not treat an upstream hash as independent proof of factual correctness.

12.3 ProductLoop

The ProductLoop provenance bridge validates canonical timestamps, receipt hashes, sequence order, and per-run continuity. Stable identities are derived from run and sequence position so divergent histories collide rather than silently fork.

ProductLoop receipts prove canonical hash continuity, not author identity. ProductLoop and Qarinah remain separate stores without a cross-ledger transaction.

12.4 Dependency direction

The integrations preserve clear ownership:

```
Cockroach SourceRecord -> Qarinah evidence
ProductLoop RuntimeEvent -> Qarinah provenance
Maqam guarded call -> Qarinah query or approved append
Qarinah verified event chain -> OKF / graph / index / Markdown / context pack
```

No adjacent package becomes a hidden dependency of the Qarinah runtime.

13. Security model

13.1 Threats considered

The current implementation is designed to detect or reduce:

1. silent capture enabled by a committed configuration;

- policy drift after trust was granted;
- event alteration, deletion, truncation, duplication, or reordering;
- rollback to an older valid log prefix;
- event-ID replay with different content;
- stale or malicious derived indexes;
- path escape through symbolic links or junctions;
- overwrite of unowned OKF output;
- oversized, deeply nested, accessor-bearing, prototype-bearing, or non-JSON input;
- prompt injection embedded in retrieved content;
- unbounded context disclosure;
- forged Maqam dispatch context;
- divergent ProductLoop sequence history; and
- accidental retention of unreviewed content in metadata mode.

13.2 Controls

The implementation uses:

- strict versioned contracts;
- canonical serialization;
- bounded strings, collections, nesting, events, logs, scans, and output packs;
- root-bound real paths;
- linked-path and multiple-link rejection;
- renewable append locks;
- flush-before-checkpoint append order;
- atomic derived replacements;
- hash-chain verification;
- machine-local rollback checkpoints;
- deterministic rebuilds;
- stale-view refusal;
- bounded best-effort redaction;
- explicit evidence coverage;
- separate read and write capabilities; and
- treatment of all retrieved content as untrusted data.

13.3 Residual risk

The current alpha has known limits:

1. machine trust is local state and is not backed by a hardware key or external transparency log;
1. the lock is not a distributed coordination protocol;
1. pattern redaction cannot recognize every secret;
1. lexical and graph retrieval do not prove semantic equivalence;
1. retention expiry filters disclosure but does not rewrite append-only history;
1. lifecycle hooks are observability surfaces, not complete host enforcement;
1. adjacent evidence ledgers do not share an atomic transaction; and
1. a privileged operating-system supervisor requires a separate implementation and threat model.

These limits are part of the release boundary, not hidden exceptions.

14. Evaluation methodology

Qarinah commits deterministic evaluators and machine-readable expected results to the repository. The public headline result comes from the software-task context benchmark, while the long-document and retrieval-regression fixtures test complementary behavior.

14.1 Portable token estimator

All reported estimated tokens in this paper use:

```
estimatedTokens = ceil(characters / 4)
```

This estimator is portable and reproducible. It is not a provider tokenizer and does not represent an OpenAI, Anthropic, Codex, or Claude usage receipt.

14.2 Software-task context benchmark

The evaluator creates 240 retained project-history records and runs six scenarios:

1. React accessibility editing;
2. a database schema migration;
3. a repository-wide TypeScript refactor;
4. web research carried into implementation;
5. production regression debugging; and
6. governed release preparation.

Both compared paths receive the same current-task source snippets. The baseline additionally receives the complete retained project history. The Qarinah path receives a cited pack compiled for the query.

The evaluator asserts:

1. every required decision ranks in the top five;
1. every returned query has direct evidence coverage;
1. no model-written summary item is selected;
1. token arithmetic matches the committed character counts; and
1. the machine-readable result remains unchanged unless the evidence is deliberately updated.

14.3 Long-document benchmark

The evaluator constructs a deterministic 384-section synthetic operations handbook with eight answer-bearing passages distributed across the beginning, middle, and end. It runs:

1. eight exact queries;
1. eight typo-tolerant queries; and
1. four unsupported controls.

Every positive query receives the same fixed 600-token ceiling. The evaluator does not tune the budget per question.

14.4 Retrieval-regression fixture

A separate 54-record fixture checks:

1. exact retrieval;
1. typo tolerance;
1. one-hop graph evidence;
1. explicit conflict recall;
1. supersession precision;
1. pack-size regression; and
1. local query timing.

This smaller fixture is a regression suite, not the public context-volume headline.

14.5 Runtime benchmark

The repository also benchmarks deterministic local append, replay, build, and query operations over a fixed retained workspace. These measurements identify implementation regressions. They are not presented as end-to-end model speed or "coding faster" results because model latency, tool execution, user review, and task difficulty are outside that measurement.

15. Results

15.1 Software-task context volume

Scenario	Full-history baseline	Qarinah context	Reduction
React accessibility edit	73,765 estimated tokens	1,025 estimated tokens	98.61%
Database schema migration	73,703	968	98.69%
Repository-wide TypeScript refactor	73,628	895	98.78%
Web research to implementation	73,693	963	98.69%
Production regression debugging	73,697	954	98.71%
Governed release preparation	73,627	877	98.81%
Weighted total	**442,113**	**5,682**	**98.71%**

The weighted result is:

```
1 - (5,682 / 442,113) = 0.987148...
```

Rounded to two decimal places, Qarinah uses **98.71% less estimated input context** than the named full-history baseline. The equivalent compression ratio is:

```
442,113 / 5,682 = 77.81:1
```

The benchmark sends 436,431 fewer estimated input-context tokens. Under a flat price of $1 per million uncached input tokens, the compared input-context slice moves from $0.4421 to $0.0057, which is **98.71% lower input-context cost at the same token rate**.

This cost translation is arithmetic over the measured context slice. It excludes output tokens, cached-input discounts, tool calls, retrieval work, fixed provider fees, and any provider-specific billing rules. It is not a claim of 98.71% lower total application cost.

15.2 Long-document retrieval

Measurement	Result
Source size	139,001 characters
Portable token estimate	34,751
Positive queries	16
Answer-bearing passage ranked first	16 / 16
Answers preserved in cited excerpts	16 / 16
Average Qarinah pack	534 estimated tokens
Largest Qarinah pack	556 estimated tokens
Worst-case estimated context reduction	98.4%
Unsupported controls rejected	4 / 4
Model-written summary items	0

This result shows targeted retrieval from a large pre-segmented source under a fixed context ceiling. It does not demonstrate native PDF ingestion, whole-book summarization, or universal question answering.

15.3 Retrieval regression

Measurement	Result
Recall@5	1.0
Mean reciprocal rank	1.0
Conflict recall	1.0
Supersession precision	1.0
Average emitted pack	2,237 characters
Raw event-log replay per query	44,364 characters
Character reduction	94.96%

All four fixed targets remain retrievable. The selected pack is larger than the earlier context-pack format because v2 includes explicit evidence-coverage metadata.

16. Interpretation

The results support four conclusions about the tested implementation.

First, accumulated project history can be separated from current-task source material and replaced with a much smaller cited pack. Second, the selected evidence can retain the required decision targets without relying on model-written summaries. Third, unsupported questions can fail closed when the caller requires direct coverage. Fourth, citations, conflict handling, and coverage metadata can fit inside a strict output budget rather than being added after selection.

The results do **not** establish:

1. 98.71% fewer tokens for every repository or task;

1. 98.71% lower total provider cost;
1. faster coding by a particular percentage;
1. improved answer quality for every model;
1. complete semantic recall; or
1. superiority over every memory or retrieval system.

Those questions require provider-reported token usage, task-success evaluation, latency measurement, quality review, ablation studies, and a broader held-out corpus.

The appropriate public statement is therefore precise:

> *Qarinah reduced estimated repeated project context from 442,113 to 5,682 tokens in the committed software-task evaluation - 98.71% less context and 77.81:1 compression - while every required target ranked in the top five with direct evidence coverage.*

17. Reproducibility

Use a maintained Node.js 22, 24, or 26 release and a reviewed source checkout:

```
npm ci
npm run check
```

Run the individual evaluations:

```
npm run evaluate:software-tasks
npm run evaluate:long-document
npm run evaluate:context
npm run benchmark
```

The relevant evidence is committed at:

1. `bench/fixtures/software-task-scenarios.mjs`;
1. `bench/results/software-task-context-0.1.0-alpha.3.json`;
1. `bench/results/long-document-context-0.1.0-alpha.3.json`;
1. `bench/results/context-evaluation-0.1.0-alpha.3.json`;
1. `scripts/evaluate-software-tasks.mjs`;
1. `scripts/evaluate-long-document.mjs`;
1. `scripts/evaluate-context.mjs`; and
1. `scripts/verify-benchmark-evidence.mjs`.

The evidence verifier recomputes public percentages and rejects unsupported headline claims. Documentation checks verify local links, architecture-source integrity, and encoding. The complete package check also rebuilds both host plugins, exercises MCP transport, runs the test and type matrices, executes every benchmark, and performs a dry-run package build.

To verify the storage model in a disposable initialized workspace:

1. record a bounded event;
2. run `qarinah doctor`;
3. build the derived views;
4. delete the graph, index, and Markdown projections;
5. rebuild them; and
6. compare the results against the same verified event head.

The projections should be reproducible while the JSONL ledger remains authoritative.

18. Release status and eligibility

No standards body, academic venue, or regulator grants general permission to publish a software white paper. A project is ready to publish one when the paper accurately identifies its category, makes traceable claims, provides enough implementation detail to be useful, and gives readers a way to reproduce or challenge the evidence.

Qarinah meets that threshold for an **implementation-backed technical white paper** because:

1. the described system exists in working source;
1. the architecture and contracts are versioned;
1. the benchmark fixtures and expected outputs are committed;
1. the headline arithmetic is machine-checked;
1. security boundaries and residual risks are documented;
1. reproduction commands are public;
1. the project has an Apache-2.0 license; and
1. the paper distinguishes measured results from open research questions.

The paper must not be represented as peer-reviewed academic research, independent third-party validation, a provider invoice, or a universal performance guarantee. The implementation remains an alpha and should be identified as such wherever the paper is distributed.

Recommended publication metadata:

1. **Document type:** Technical white paper
1. **Title:** *Qarinah: Less Context. More Proof.*
1. **Subtitle:** *An evidence-linked project-memory compiler for coding agents*
1. **Author:** Ajnas NB
1. **Implementation version:** `0.1.0-alpha.3`
1. **Paper version:** 1.0
1. **License:** Apache-2.0
1. **Canonical source:** this repository at one reviewed commit

1. **Evidence:** the machine-readable files and commands listed in Section 17
1. **Review status:** implementation-backed, not peer-reviewed

The release tag, paper, package, generated plugins, benchmark results, and architecture image should all point to the same reviewed commit. If code or evidence changes, the paper version should advance rather than silently changing an archived release.

19. Open-source model and stewardship

Qarinah is licensed under Apache-2.0. That permits broad use, modification, redistribution, and commercial use under the license terms while preserving copyright and notice obligations. The open license makes the implementation auditable and allows host integrations to be inspected before they receive access to project events.

Open source does not by itself protect a product idea from commercial competition. Long-term differentiation must come from implementation quality, trust, interoperability, distribution, community, and the discipline of publishing evidence that users can reproduce.

Security vulnerabilities should be reported privately according to the repository security policy. Behavioral changes to capture, disclosure, trust, event contracts, or governance boundaries require explicit review and migration notes.

20. Roadmap and research agenda

The path to a stable release includes:

1. at least 100 held-out positive and negative retrieval queries;
1. paraphrase, typo, conflict, supersession, time, authority, and unsupported-query coverage;
1. provider-reported Codex and Claude token measurements under matched models and tools;
1. task-success, unsupported-answer, latency, and cost review;
1. ablations for lexical, trigram, graph, diversity, conflict, and coverage components;
1. language-aware project adapters behind versioned contracts;
1. signed context-pack manifests;
1. signed or independently anchored ledger checkpoints;
1. explicit deletion and retention workflows;
1. stronger multi-host coordination;
1. independent threat-model review; and
1. a separately designed privileged supervisor if the wider product evolves toward cross-platform operating-system mediation.

Qarinah itself should remain the evidence-linked memory and context layer. A future operating-system control plane may use its record, but privileged process, filesystem, network, identity, secret, and device controls require separate platform-specific enforcement.

21. Conclusion

Coding agents need continuity, but continuity should not require replaying everything or trusting an opaque summary.

Qarinah keeps durable evidence and task-time context as two different artifacts. The append-only ledger preserves the permitted project record. Deterministic projections make that record searchable and inspectable. The context compiler selects a small cited working set under an explicit budget. Conflicts, supersession, authority, retention, and evidence coverage remain visible rather than being compressed away.

The current alpha demonstrates that this design works end to end across local storage, project structure, retrieval, Codex and Claude Code adapters, MCP diagnostics, Maqam governance, crawler evidence, workflow provenance, and portable OKF export. Its committed evaluations show large context-volume reductions while preserving the required evidence in the tested scenarios.

The central promise is intentionally simple:

> *Less context. More proof.*

Appendix A. Claim-to-evidence matrix

Public claim	Evidence	Qualification
98.71% less estimated context	Software-task result: 442,113 to 5,682 estimated tokens	Compared with the named full-history baseline using `ceil(characters / 4)`
77.81:1 context compression	Same committed software-task result	Ratio of total estimated input-context tokens
98.71% lower input-context cost at the same token rate	Arithmetic over the same measured context slice	Not total provider or application cost
Every required target ranked in the top five	Six committed software-task results	Applies to the committed scenarios
Direct evidence coverage for every software-task query	Context-pack v2 output assertions	Lexical retained-evidence coverage, not answer correctness
No model-written summary items	Software-task and long-document assertions	Does not prohibit retaining a summary as a separately labeled event
At least 98.4% estimated reduction on the long document	Largest fixed-budget pack compared with the complete synthetic source	Pre-segmented deterministic source; not native PDF ingestion
Unsupported controls failed closed	Four long-document controls with direct coverage required	Callers permitting partial coverage choose a weaker policy
Local-first and no Qarinah API key	Runtime architecture and package dependencies	AI hosts and external sources may still require their own access
Deterministic rebuildable graph, index, Markdown, and OKF	Source implementation and test suite	Reproducibility depends on the same verified event head and build inputs

Appendix B. Artifact map

Topic	Canonical project document
System architecture	ARCHITECTURE.md
Benchmark methodology and results	BENCHMARKS.md
Security model	SECURITY.md
Host integrations	HOST-INTEGRATIONS.md
Maqam, crawler, ProductLoop, and OKF boundaries	INTEROPERABILITY.md
Migration notes	MIGRATIONS.md
Release gates	LAUNCH.md
Public package entry point	README.md

Appendix C. Suggested citation

```
Ajnas NB. "Qarinah: Less Context. More Proof. An evidence-linked
project-memory compiler for coding agents." Technical white paper,
version 1.0, July 2026. Qarinah 0.1.0-alpha.3.
```